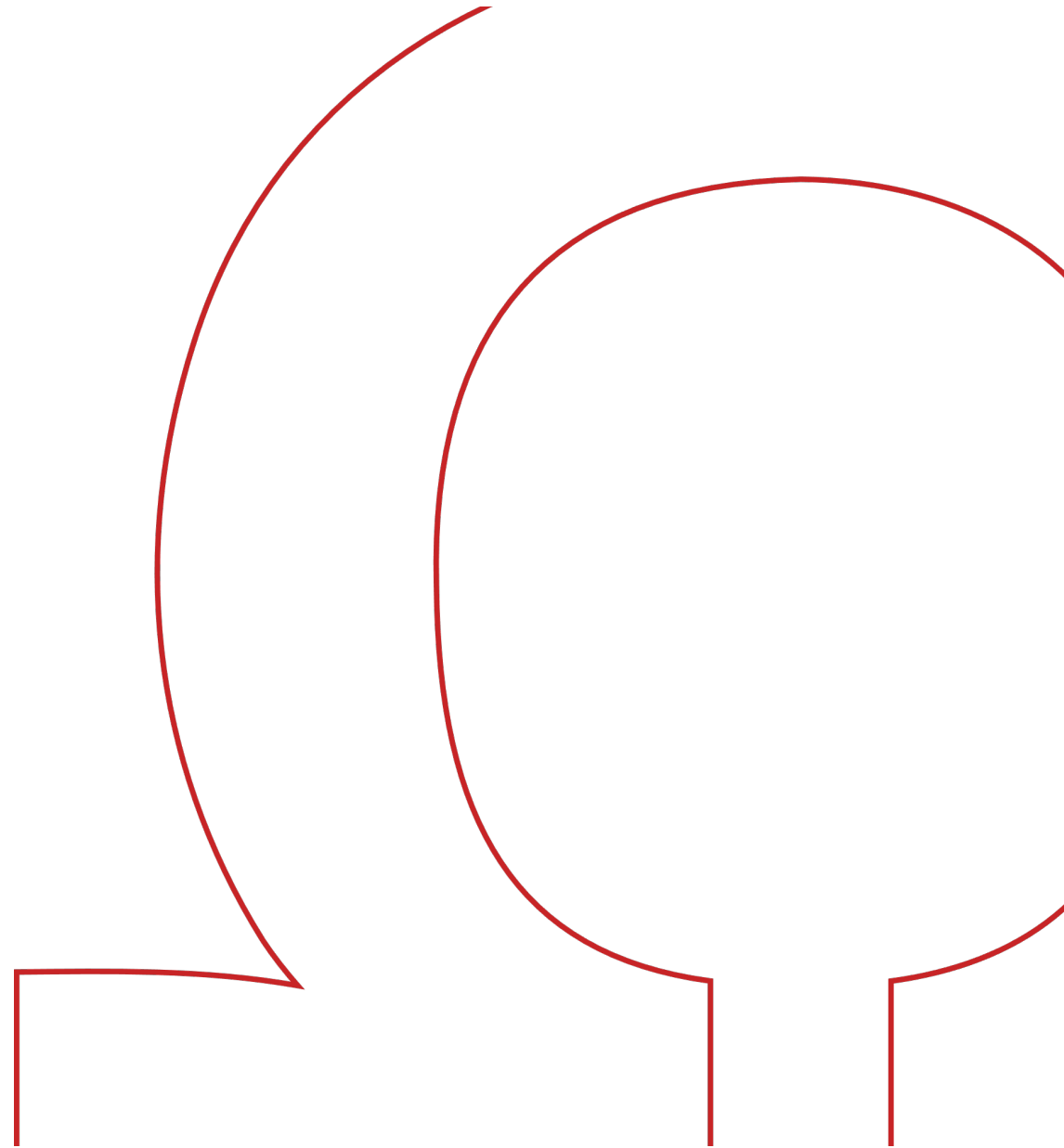


Prog III / A

Software Engineering



Software- Qualitätssicherung

Software-Qualitätssicherung

Definition "Qualität" (DIN EN ISO 9000:2015)

Qualität [ist der] Grad, in dem ein Satz inhärenter
Merkmale [des betrachteten Produkts] die
Anforderungen [an das Produkt] erfüllt.

Software-Qualitätssicherung

Definition "Qualität" (DIN EN ISO 9000:2015)



Ist das ein qualitativ hochwertiges Produkt?



..das hängt von den Anforderungen ab!

Software-Qualitätssicherung

Definition "Qualität" (DIN EN ISO 9000:2015)

Qualität ist somit eine **relative Größe**, die sich immer aus einem **Vergleich** ergibt. Bei einem Vorgehen nach dem Wasserfallmodell könnte z.B. verglichen werden:

- Entspricht das Lastenheft den Wünschen und Vorstellungen der Anwender?
- Sind im Pflichtenheft alle Anforderungen des Lastenhefts in korrekter Weise behandelt?
- Ist das IT-Konzept eine vernünftige technische Spezifikation der Vorgaben des Pflichtenhefts?
- Ist der implementierte Code konform mit dem IT-Konzept?
- ...

Software-Qualitätssicherung

Konstruktive und analytische Maßnahmen

Konstruktive Qualitätssicherung

Die konstruktive Qualitätssicherung umfasst alle Methoden und Techniken, die a priori sicherstellen, dass ein Software-System den geforderten Anforderungen genügt. Im Besonderen fallen hierunter alle Maßnahmen, die entwicklungsbegleitend zur Anwendung kommen und dem Software-Entwickler damit die Möglichkeit eröffnen, Fehler frühzeitig zu erkennen oder gar von vorne herein zu vermeiden.

Beispiele: Schulungen, Checklisten, Vorgehensmodelle, Clean Code, Pair Programming, Werkzeuge, ...

Analytische Qualitätssicherung

Die analytische Qualitätssicherung konzentriert sich darauf, das tatsächliche Qualitätsniveau der Software a posteriori zu messen. Es wird untersucht, wie präzise und umfassend das Software-System den vorab definierten Anforderungen entspricht, ob die Software effizient arbeitet, frei von Fehlern ist und den berechtigten Erwartungen der Benutzer gerecht wird.

Beispiele: Reviews, Audits, Code-Inspektionen, statische Codeanalyse, dynamische Codeanalyse, manuelle Softwaretests, automatisierte Softwaretests

Analytische Qualitätssicherung

Analytische Qualitätssicherung

Statische Codeanalyse

Im Rahmen einer statischen Codeanalyse werden Metriken aus dem Quellcode berechnet.

The screenshot shows a static code analysis tool interface. On the left, a 'MetricsTree' sidebar lists the class 'Zahlenfilter' and its methods: 'Zahlenfilter(int)', 'main(String[])', and 'printNumbers()'. The 'printNumbers()' method is expanded, showing various metrics with green and red circular icons. The main panel, titled 'Class: Zahlenfilter', displays a table of metrics. The table has columns for 'Me...', 'Met...', 'Description', 'Value', and 'Regular...'. The metrics listed include NOC (Number Of Children), DIT (Depth Of Inheritance Tree), LCOM (Lack Of Cohesion Of Methods), RFC (Response For A Class), WMC (Weighted Methods Per Class), NCSS (Non-Commenting Source Statements), and several Halstead metrics (Errors, Length, Vocabulary, Volume).

Me...	Met...	Description	Value	Regular...
○ NOC	Chidamb...	Number Of Children	0	[0..2)
○ DIT	Chidamb...	Depth Of Inheritance Tree	1	[0..3)
○ LCOM	Chidamb...	Lack Of Cohesion Of Methods	1	
○ RFC	Chidamb...	Response For A Class	5	[0..45)
○ WMC	Chidamb...	Weighted Methods Per Class	9	[0..12)
○ NCSS	Chr. Cle...	Non-Commenting Source Statements	22	[0..1000)
○ CHER	Halstead...	Halstead Errors	0,1957	
○ CHEF	Halstead...	Halstead Effort	14227,66...	
○ CHD	Halstead...	Halstead Difficulty	27,6	
○ CHVC	Halstead...	Halstead Vocabulary	43	
○ CHVU	Halstead...	Halstead Volume	515,4052	

Analytische Qualitätssicherung

Statische Codeanalyse - Metriken

Eine (kleine) Auswahl an Metriken, die durch statische Codeanalyse ermittelt werden können:

- Anzahl der Codezeilen (***Source lines of Code – SLOC***)
- Anzahl der unmittelbaren Unterklassen einer Klasse (***Number of Children – NOC***)
- Summe der Operatoren und Operanden in einem Codestück (***Halstead-Länge***)
- ...

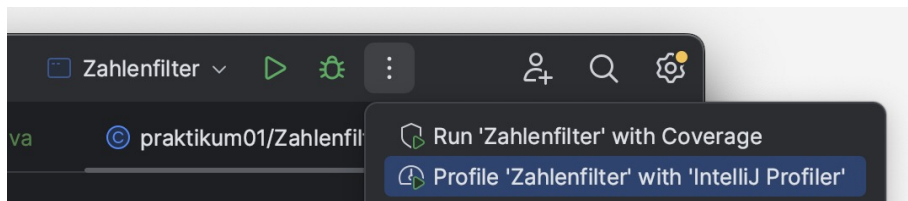
Durch Kombination der unterschiedlichen Metriken (z.B. Halstead-Länge im Verhältnis zu SLOC) entstehen aussagekräftige ***Kennzahlen***, die Auskunft über den Zustand des Codes geben können.

Auch ist es möglich, für jede Kennzahl einen Korridor festzulegen, in dem sie sich bewegen darf.

Analytische Qualitätssicherung

Dynamische Codeanalyse - Profiler

Bei der dynamischen Codeanalyse kommen häufig so genannte **Profiler** zum Einsatz. Sie ermitteln z.B. Anzahl der Aufrufe einer bestimmten Methode oder die Rechenzeit, die für diese Methode aufgewendet wird.



Profiler		Home	Zahlenfilter 10.11.23, 16:39			
Flame Graph		Call Tree	Method List	Timeline	Events	
All threads merged			Method ^	Execution time(ms)		
RMI TCP Accept-0	34.0%		jdk.jfr.internal.WriteableUserPath\$1.run()			24
DestroyJavaVM	33.0%		jdk.jfr.internal.WriteableUserPath.doPrivilegedIO(Callable)			24
JFR Shutdown Hook	24.7%		praktikum01.Zahlenfilter.main(String[])			8
RMI TCP Connection(1)-192.168.0.109	8.2%		praktikum01.Zahlenfilter.printNumbers()			8
			sun.launcher.LauncherHelper.checkAndLoadMain(boolean, int, String)			8
			sun.launcher.LauncherHelper.loadMainClass(int, String)			8

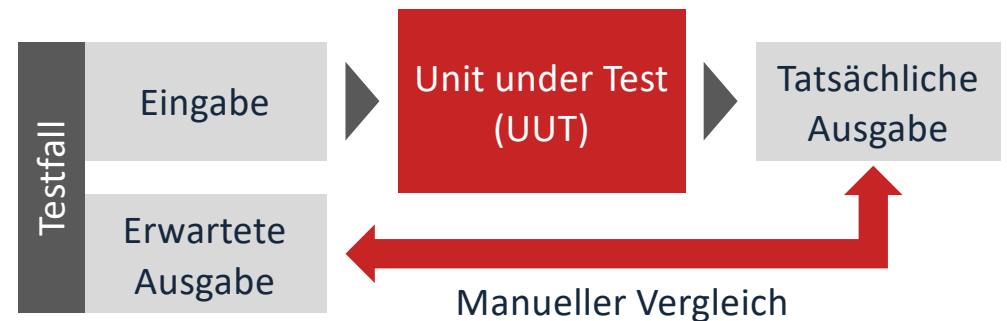
Analytische Qualitätssicherung

Dynamische Codeanalyse – Black-Box-Test

Während die bisherigen Maßnahmen zur analytischen Qualitätssicherung stets Kenntnis des Quellcodes vorausgesetzt haben (**White-Box-Test**), wird bei einem Black-Box-Test das Software-System bzw. die **Unit under Test** ausschließlich mittels der öffentlichen Schnittstelle getestet.

Im einfachsten Fall kann dies bedeuten, dass ein Tester manuell Daten eingibt und dann überprüft, ob das gewünschte Ergebnis ausgegeben wird.

Die Eingabe zusammen mit der erwarteten Ausgabe wird auch als **Testfall** bezeichnet.



Analytische Qualitätssicherung

Dynamische Codeanalyse – Black-Box-Test

Eine besondere Herausforderung stellt die Auswahl der Testfälle dar, da normalerweise nicht alle Eingaben, die prinzipiell möglich wären, getestet werden können.

Hilfreich für die Reduktion der Zahl von Testfällen sind:

- das Testen von Grenzwerten oder Ausnahmesituationen
- das Testen von einem Fall für jede fachliche Ergebnisvariante
(die übrigen Eingaben, die zur gleichen Ergebnisvariante führen, werden als äquivalent betrachtet, daher auch **Äquivalenzklassentest**)

Dennoch ist bei einer komplexen Software mit einer sehr großen Zahl von Testfällen zu rechnen.

Analytische Qualitätssicherung

Dynamische Codeanalyse – Black-Box-Test



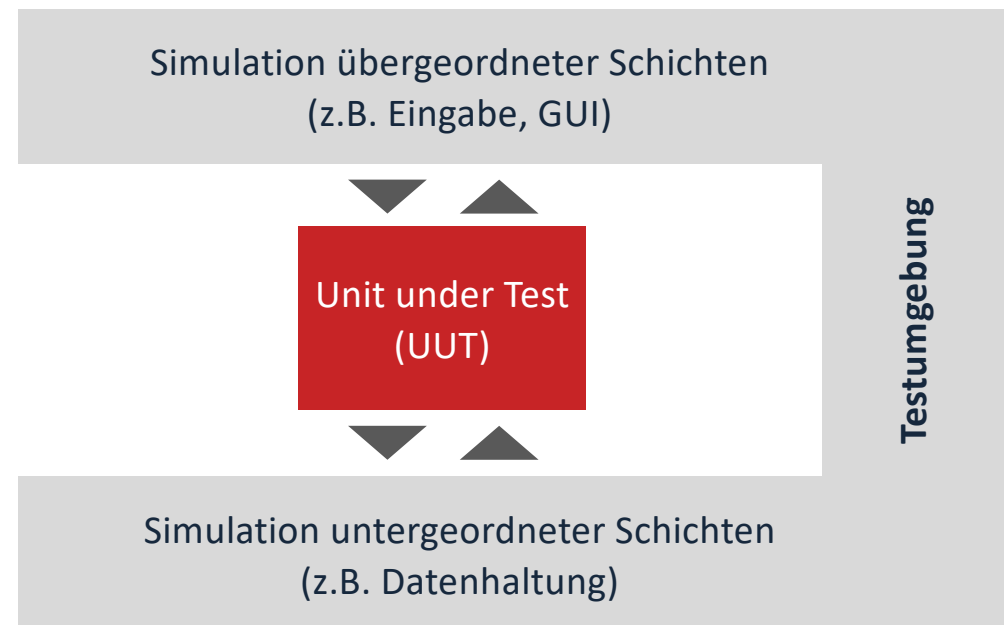
Analytische Qualitätssicherung

Dynamische Codeanalyse – Automatisierter Softwaretest

Beim **automatisierten Softwaretest** werden die Testfälle programmiert (**Testcode**) und in einer **Testumgebung** ausgeführt.

Tests können so sehr **einfach wiederholt** werden, sind **schnell** und **kostengünstig**.

Außerdem stellen die automatisierten Testfälle eine **implizite Dokumentation** der Unit under Test dar.



Analytische Qualitätssicherung

Grundprobleme von a posteriori Maßnahmen

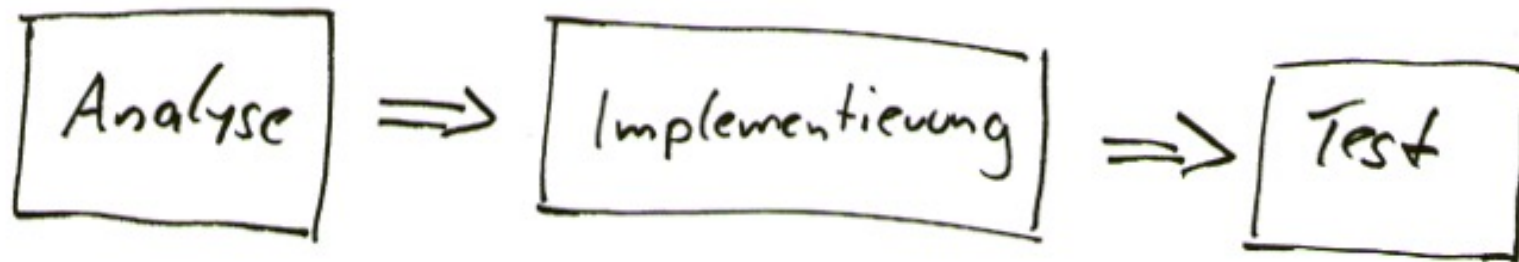
Man kann keine Qualität in Software hineinprüfen. Qualität kann man nur in Software hineinkonstruieren.

Test können Fehler nachweisen, nicht aber die Fehlerfreiheit.

Test Driven Development (TDD)

Test Driven Development

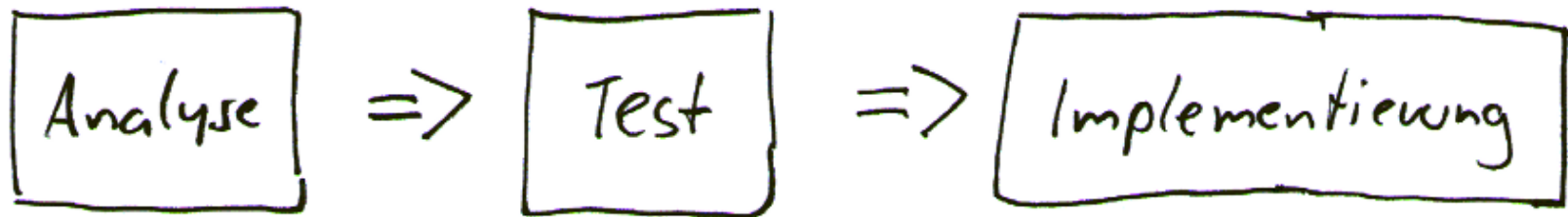
Wo ist das Problem?



Bei einer klassischen Vorgehensweise findet der Test erst nach der Implementierung statt.

Test Driven Development

Test First!



Idee: Wir schreiben erst den Testcode, und erst anschließend den Produktivcode

Test Driven Development

Die drei Gesetze von TDD

1

Sie dürfen Produktionscode erst schreiben, wenn Sie einen scheiternden Test geschrieben haben.

2

Der Test darf nicht mehr Code enthalten, als für das Scheitern und ein korrektes Kompilieren des Tests erforderlich ist.

3

Sie dürfen nur so viel Produktionscode schreiben, wie für das Bestehen des gegenwärtig scheiternden Tests ausreicht.

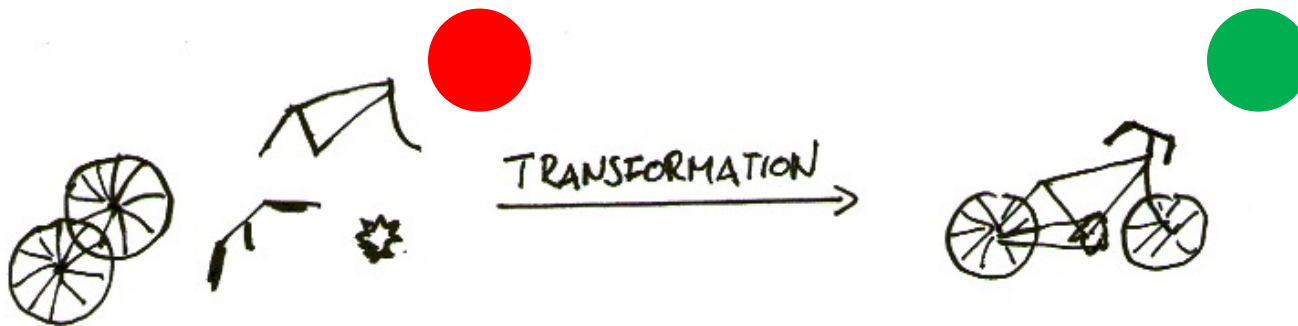
Test Driven Development

Red-Green-Refactor – Zyklus



Test Driven Development

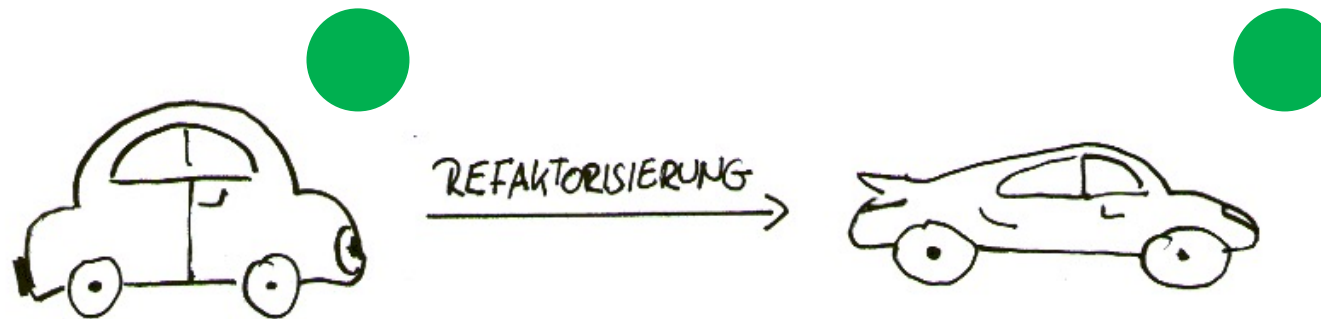
Transformation



Bei einer **Transformation** wird Code ergänzt und zusammengesetzt, so dass eine neue Funktionalität entsteht.

Test Driven Development

Refaktorisierung



Unter einer **Refaktorisierung** (auch: **Refactoring**) versteht man eine Codeänderung, die ausschließlich der Verbesserung der Codequalität dient, ohne die Funktionalität des Codes zu verändern.

Entkoppelung von Klassen

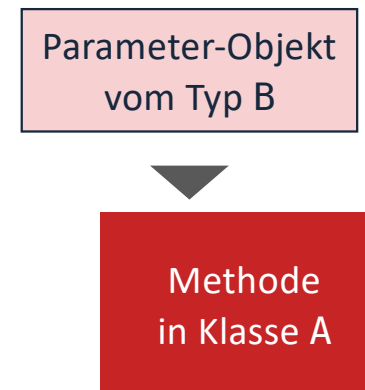
Entkoppelung von Klassen

Dependency Injection

Häufige Methodenaufrufe zwischen Klassen machen spätere Änderungen und Entflechtungen schwierig.

Daher wird oft eine Instanz der jeweils anderen Klasse als Parameter übergeben, über die dann die Methoden der anderen Klasse aufgerufen werden.

Ein solches Vorgehen heißt ***Dependency Injection*** und ist "best practice" zur Entkoppelung von Klassen.



Entkoppelung von Klassen

Beispiel Dependency Injection

```
class A
{
    public void methodeVonA(B b) {
        b.tuwas();
    }
}
```



```
class B
{
    public void tuwas() {
        ...
    }
}
```

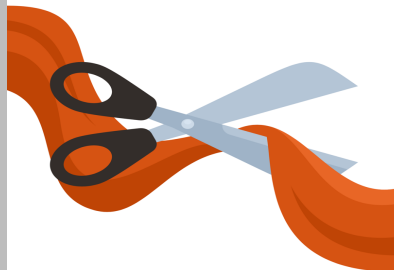
In diesem Beispiel sind die Klassen **nicht** vollständig voneinander entkoppelt. Klasse A nutzt Klasse B explizit als Typ bei den Argumenten einer Methode.

Somit kann Klasse A nicht ohne Klasse B genutzt werden.

Entkoppelung von Klassen

Beispiel Dependency Injection (verbessert)

```
class A
{
    public void methodeVonA(I i) {
        i.tuwas();
    }
}
```



```
Interface I
{
    void tuwas();
}
```



```
class B implements I
{
    public void tuwas() {
        ...
    }
}
```

Jetzt wird ein **Interface** als Typ für das Argument der Methode verwendet. Die Klasse A ist nicht mehr mit der Klasse B gekoppelt.

Es kann mehr als eine Implementierung des Interface I geben.

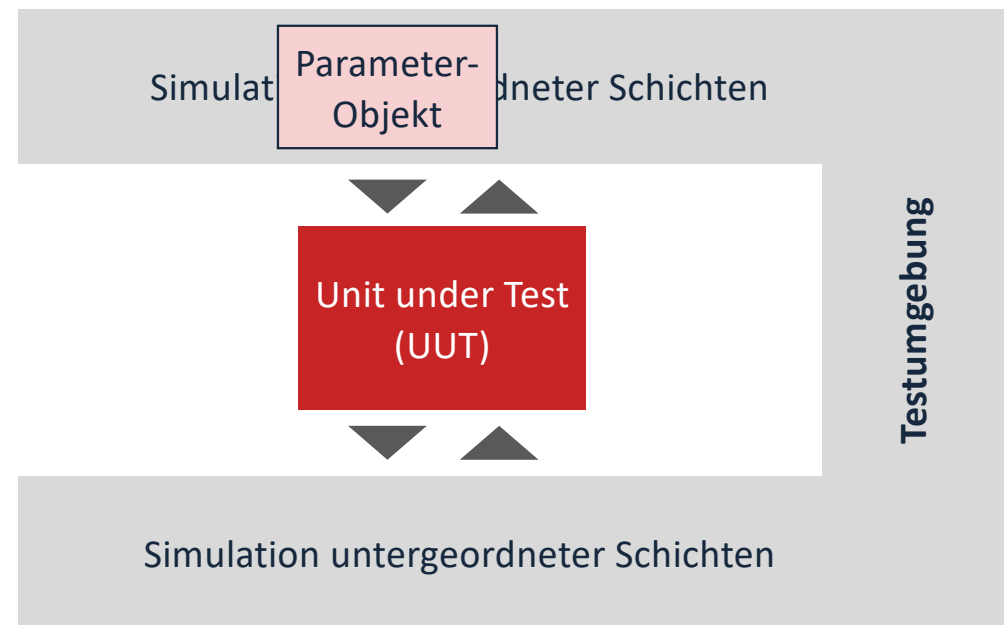
Mocks und Stubs

Mocks und Stubs

Automatisierter Test bei Dependency Injection

Es werden beim Aufruf der zu testenden Funktion in der **Unit under Test** Objekte als Parameter übergeben, bei denen dann im Rahmen der Ausführung Methoden aufgerufen werden.

Die Überprüfung dieser Methoden ist nicht Gegenstand des Tests. Auch ist die Ausführung dieser Methoden u.U. nicht gewünscht (z.B. wegen einer möglichen Manipulation von Daten in der Datenbank).



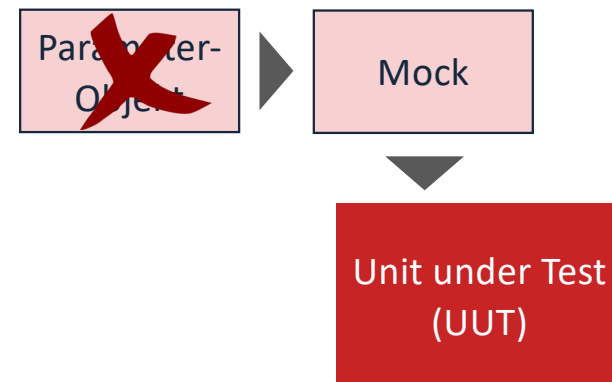
Mocks und Stubs

Mocking

Um das Verhalten der **Unit under Test** zu isolieren, werden injizierte Parameter durch **Mocks** ersetzt. Dies ist nützlich, wenn die echten Parameter-Objekte nicht in den Test integriert werden können oder sollen.

Voraussetzung:

Der Datentyp des Parameters ist durch ein Interface gegeben, das sowohl das vom echten Parameter-Objekt als auch vom Mock implementiert wird.

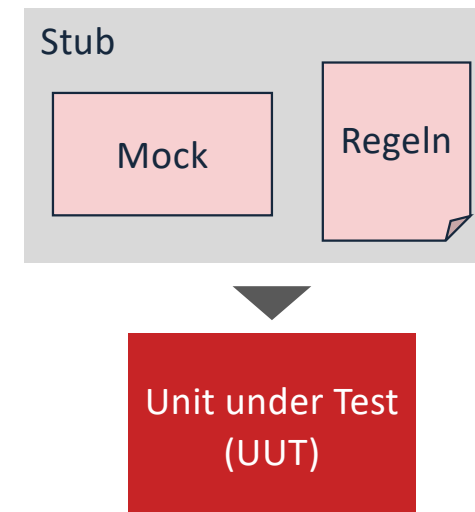


Mocks und Stubs

Stubbing

Ein **Stub** ist eine besondere Form eines **Mock**. Hier werden vor der Nutzung Regeln hinterlegt, welche Werte das Stub später bei Aufrufen im Rahmen des Tests zurückliefern soll.

Auf diese Weise können komplexere Testfälle wiederholbar definiert und ausgeführt werden.



Vielen Dank für die Aufmerksamkeit!

Fragen?